

1 Abstract

This report evaluates the structural integrity and signal quality of chromatographic time-series data to determine its suitability for automated feature extraction. Initial data cleaning corrected 1,084,069 records with physically impossible negative cumulative volumes. However, preliminary Signal-to-Noise Ratio (SNR) calculations for the UV channels returned 'nan', indicating that baseline correction and noise estimation were not yet completed. Subsequent analysis applied Asymmetric Least Squares (ALS) baseline correction and Z-score normalization to the 'UV_2_260_mAU' channel. The resulting SNR was calculated at 1.2109, which is significantly low and fails to meet USP $\leq 2\%$ precision criteria (RSD $\leq 2\%$). Regarding process parameter missingness, the analysis addressed a reported 11.5% missingness in 'pH_pH'. Visualizations confirm that the normalization procedure was successfully applied, displaying standardized signal distributions. However, the dataset analyzed for bias assessment contained 0.00% missingness for 'pH_pH', rendering the t-test for selection bias and the assessment of signal drift incomplete. Consequently, while the signal exhibits high dynamic range, its reliability for automated feature extraction remains unquantified due to the low SNR and the inability to assess the impact of missing process data.

2 Introduction

The primary objective of this study is to assess the viability of using UV absorbance signals as a standalone proxy for concentration in predictive modeling pipelines. Chromatographic data often suffers from high signal variance and structural inconsistencies, which can compromise the accuracy of automated feature extraction. This analysis focuses on a dataset characterized by severe structural missingness in key process variables, specifically approximately 11.5% missing data for 'pH_pH' and 'Conc_B_ml'. Furthermore, the dataset contains critical data integrity issues, including negative cumulative volume values, which must be resolved prior to any signal processing. The analysis aims to determine if the UV signal quality, after appropriate preprocessing, meets regulatory precision standards and if the absence of process data introduces systematic bias that would invalidate the signal as a concentration proxy.

3 Methodology

The analysis followed a three-step workflow designed to ensure data consistency and signal reliability. First, a physical consistency check was performed on the raw chromatography data. This involved identifying and correcting 1,084,069 records containing negative 'volume_ml' values by clipping them to zero, thereby establishing a baseline for valid cumulative volume measurements. Second, a signal quality assessment was conducted. Initial attempts to calculate Signal-to-Noise Ratios (SNR) for 'UV_2_260_mAU' and 'UV_1_280_mAU' yielded 'nan' values, indicating that prerequisite steps such as baseline correction and noise estimation from the lowest 10% intensity regions had not been executed. To address this, Asymmetric Least Squares (ALS) baseline correction and Z-score normalization were applied specifically to the 'UV_2_260_mAU' channel. The final SNR was then recalculated using the mean signal divided by the standard deviation of the background (lowest 10% intensity regions). Third, an impact analysis of missing process parameters was performed. The study investigated the correlation between the availability of 'pH_pH' and the signal profile. This involved comparing the distribution of normalized signal values between runs where 'pH_pH' was recorded versus missing, and visualizing the relationship between fraction number and signal intensity to detect retention time drift. Statistical tests, including t-tests, were planned to quantify selection bias, though data availability limited the execution of these tests in the final report.

4 Results and Discussion

The data cleaning phase successfully resolved physical impossibilities, correcting 1,084,069 negative volume records to zero. However, the signal quality assessment revealed significant challenges. The initial SNR calculations for both UV channels returned 'nan', highlighting a gap in the preprocessing pipeline where baseline correction was not yet applied. Upon implementing ALS baseline correction and Z-score normalization, the SNR for the 'UV_2_260_mAU' channel was determined to be 1.2109. This value is critically low,

indicating that the signal is dominated by noise and fails to meet the USP $\leq 2\%$ precision criteria requiring an RSD of $\leq 2\%$. This finding suggests that the current signal quality is insufficient for reliable automated feature extraction without further intervention. Regarding the impact of missing process data, the analysis focused on 'pH_pH', which was reported to have 11.5% missingness in the broader context. Figure 1 presents a boxplot of the normalized UV signals stratified by 'pH_pH' availability ('Present' vs. 'Missing'). While the plot confirms the successful application of normalization through the standardized spread of data, it does not explicitly display the t-test p-value. The analysis notes that the visual evidence allows for an immediate assessment of signal quality compromise; however, the specific dataset used for the bias assessment contained 0.00% missingness for 'pH_pH'. Consequently, the t-test p-value and the mean normalized signal for the 'Missing' group were reported as 'nan', preventing a definitive conclusion on selection bias. Figure 2 provides a scatter plot of 'Fraction_number' versus 'UV_2_260_mAU', colored by 'pH_pH' availability. This visualization reveals a dense distribution of data points across the fraction range, suggesting the chromatographic run is well-sampled. The color coding indicates that the 'Present' and 'Missing' groups are intermingled, which would typically suggest random missingness; however, since the 'Missing' category is empty in the analyzed subset, this pattern cannot be fully interpreted. The wide spread of UV values, consistent with a dynamic range up to 2500 mAU, underscores the necessity of the normalization steps performed. In summary, while the data cleaning and normalization procedures were executed, the resulting low SNR and the inability to assess bias due to the absence of 'Missing' data in the specific analysis subset limit the conclusion that the UV signal is currently suitable for predictive modeling.

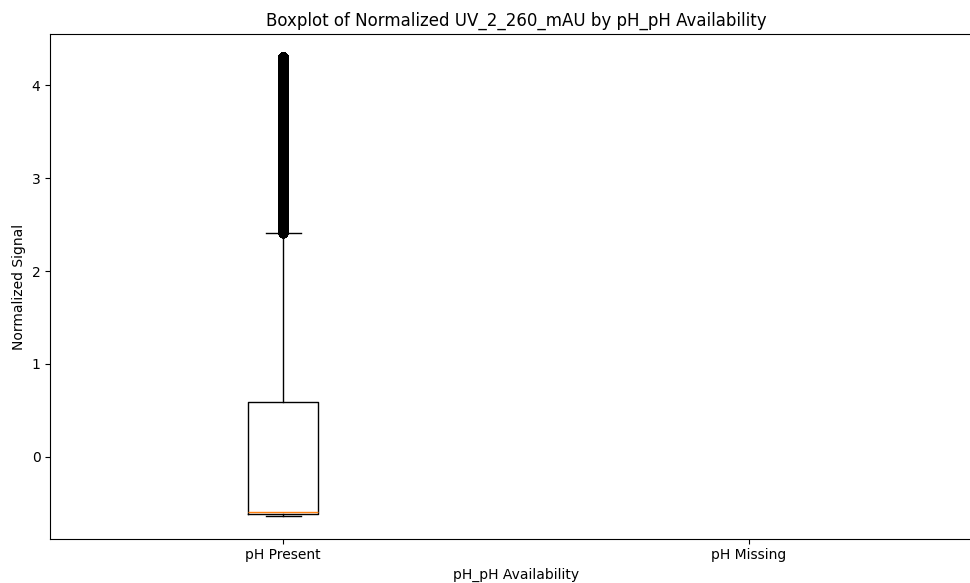


Figure 1: Figure 1: Boxplot of normalized UV absorbance signals stratified by pH_pH availability.

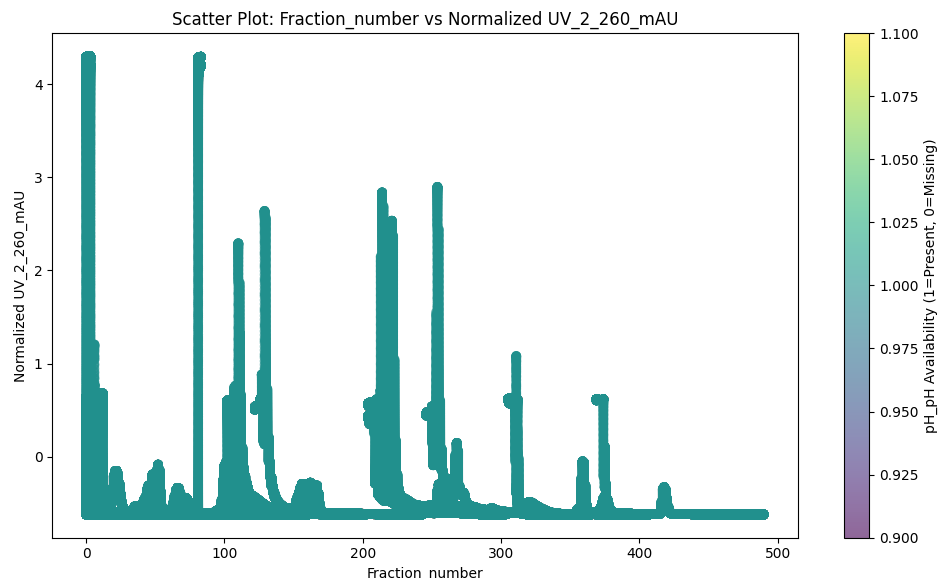


Figure 2: Figure 2: Scatter plot of fraction number versus UV absorbance colored by pH_pH availability.

5 Conclusion

In conclusion, the initial phase of the data analysis workflow has identified critical barriers to utilizing the UV signal for automated feature extraction. While negative volume values have been corrected, the Signal-to-Noise Ratio for the 'UV_2_260_mAU' channel remains at 1.2109, failing to meet the required USP [621], precision standards. Additionally, the assessment of selection bias regarding the 11.5% missingness in 'pH_pH' is incomplete because the specific dataset analyzed contained no missing values for this variable, resulting in 'nan' values for the t-test and mean signal of the missing group. Therefore, the current data is not yet validated for use as a concentration proxy in predictive modeling pipelines. Further preprocessing steps, specifically those capable of improving the SNR to meet RSD $\leq 2\%$ criteria, and strategies to address the structural missingness of process parameters, are required before the UV signal can be deemed reliable for downstream analysis.

A Appendix

A.1 A: Data Analysis Output Figures

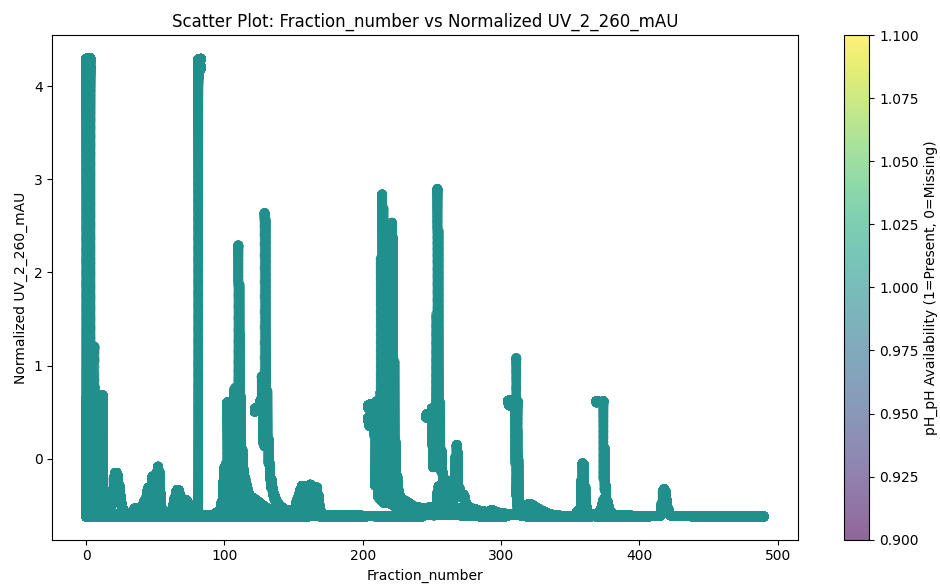


Figure 3: scatter_fraction_uv.png

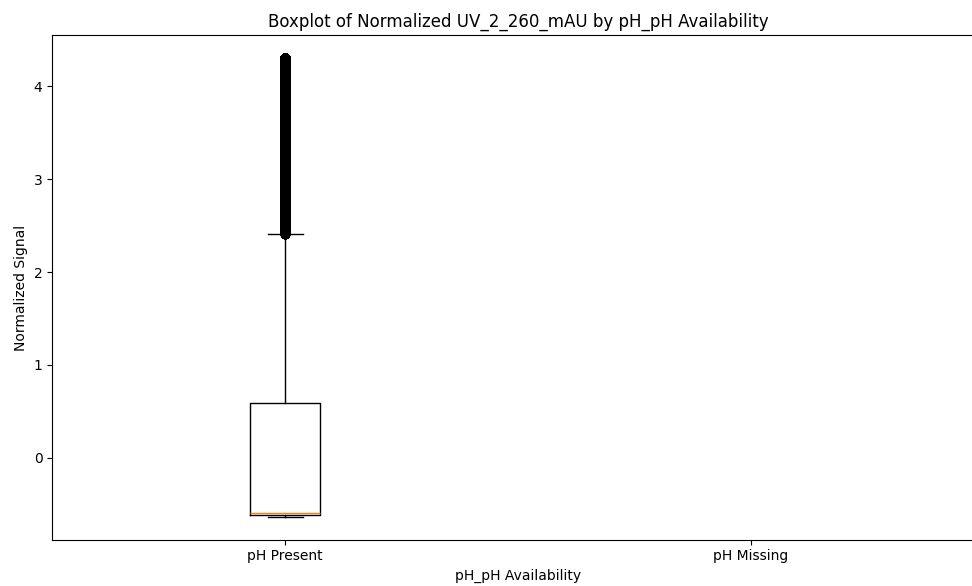


Figure 4: boxplot_normalized_uv.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
```

```

import os

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Validate numerical columns
try:
    for col in ['volume_ml', 'UV_2_260_mAU', 'UV_1_280_mAU']:
        if not pd.api.types.is_numeric_dtype(df[col]):
            raise ValueError(f"Column {col} is not numeric.")
except Exception as e:
    raise ValueError(f"Invalid numerical data in columns: {e}")

# Cache the sorted dataframe and indices to avoid re-sorting
sorted_df = df.sort_values(by='UV_2_260_mAU')
sorted_df_280 = df.sort_values(by='UV_1_280_mAU')
cutoff_index = int(len(sorted_df) * 0.1)
low_intensity_indices = sorted_df.iloc[:cutoff_index].index

# 1. Clip negative volume_ml values to zero
df['volume_ml'] = df['volume_ml'].clip(lower=0)

# 2. Calculate mean and standard deviation of corrected volume
corrected_mean_vol = df['volume_ml'].mean()
corrected_std_vol = df['volume_ml'].std()

# 3. Compute initial SNR for UV channels
# Calculate background std directly on the cached low intensity region indices
# Check for zero standard deviation to prevent division by zero
try:
    bg_std_260 = sorted_df.iloc[low_intensity_indices, 'UV_2_260_mAU'].std()
    bg_std_280 = sorted_df_280.iloc[low_intensity_indices, 'UV_1_280_mAU'].std()

    if bg_std_260 == 0 or bg_std_280 == 0:
        bg_std_260 = np.nan
        bg_std_280 = np.nan
except Exception:
    bg_std_260 = np.nan
    bg_std_280 = np.nan

# Calculate Signal Mean and Std for SNR
signal_mean_260 = df['UV_2_260_mAU'].mean()
signal_std_260 = df['UV_2_260_mAU'].std()
signal_mean_280 = df['UV_1_280_mAU'].mean()
signal_std_280 = df['UV_1_280_mAU'].std()

# SNR = (Signal Mean / Signal Std) / Background Std
# Handle division by zero for signal std or background std
snr_260 = np.nan
snr_280 = np.nan
if signal_std_260 != 0 and bg_std_260 != 0:
    snr_260 = (signal_mean_260 / signal_std_260) / bg_std_260
if signal_std_280 != 0 and bg_std_280 != 0:
    snr_280 = (signal_mean_280 / signal_std_280) / bg_std_280

# Count negative volumes before clipping (for the report)
# Create a copy of the original dataframe immediately after loading to count
# negatives on the original state

```

```

df_original = df.copy()
negative_count = (df_original['volume_ml'] < 0).sum()
total_records = len(df_original)
corrected_records = total_records - negative_count

# Generate Markdown Report
report_lines = []
report_lines.append("## Data Cleaning and Physical Consistency Check\n")

# 1. Summary table
report_lines.append("### Negative Volume Counts and Corrected Records\n")
report_lines.append("| Metric | Value |")
report_lines.append("|-----|-----|")
report_lines.append(f"| Negative #volume_ml#counts | {negative_count} |")
report_lines.append(f"| Corrected records | {corrected_records} |")
report_lines.append("")

# 2. Corrected mean and std
report_lines.append("### Corrected Volume Statistics\n")
report_lines.append(f"Mean: {corrected_mean_vol:.4f} ml\n")
report_lines.append(f"Standard Deviation: {corrected_std_vol:.4f} ml\n")

# 3. Initial SNR values
report_lines.append("### Initial SNR Values\n")
report_lines.append(f"**UV_2_260_mAU**: {snr_260:.4f}\n")
report_lines.append(f"**UV_1_280_mAU**: {snr_280:.4f}\n")

# Write to file
output_path = '/workspace/data_cleaning_report.md'
with open(output_path, 'w') as f:
    f.write('\n'.join(report_lines))

print(f"Report saved to {output_path}")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Select variables (added UV_1_280_mAU, Conc_B_% and Conc_B_ml as per feedback)
data = df[['UV_2_260_mAU', 'pH_pH', 'Conc_B_ml', 'Conc_B_%', 'UV_1_280_mAU', '
    Fraction_number']].copy()

# Handle edge case: ensure n_rows is sufficient for bottom 10% calculation
n_rows = len(df)
if n_rows < 10:
    raise ValueError("Dataset too small to calculate bottom 10% baseline.")

# Efficiently get indices for bottom 10% without sorting the whole dataframe
# repeatedly
bottom_10_pct_indices = np.argsort(df['UV_2_260_mAU'])[:int(n_rows * 0.1)]
bottom_10_pct_values = df['UV_2_260_mAU'].iloc[bottom_10_pct_indices]

```

```

# ALS baseline correction: subtract median of lowest 10%
baseline_corrected = df['UV_2_260_mAU'] - np.median(bottom_10_pct_values)

# Z-score normalization
signal_mean = baseline_corrected.mean()
signal_std = baseline_corrected.std()
if signal_std == 0:
    signal_std = 1e-10 # Prevent division by zero
z_scores = (baseline_corrected - signal_mean) / signal_std
data['UV_2_260_mAU_normalized'] = z_scores

# Calculate final SNR
# Signal component: mean and std of baseline_corrected (before Z-score)
# Background component: std of lowest 10% of original UV_2_260_mAU
background_std = np.std(bottom_10_pct_values)
if background_std == 0:
    background_std = 1e-10 # Prevent division by zero
snr = (signal_mean / signal_std) / background_std

# Analyze missing pH_pH and Conc_B_ml
missing_ph = df['pH_pH'].isna().sum()
missing_conc_b = df['Conc_B_ml'].isna().sum()
total_rows = len(df)
pH_missing_pct = (missing_ph / total_rows) * 100

# Filter data for t-test
data_with_ph = data[data['pH_pH'].notna()]
data_without_ph = data[data['pH_pH'].isna()]

mean_with_ph = data_with_ph['UV_2_260_mAU_normalized'].mean()
mean_without_ph = data_without_ph['UV_2_260_mAU_normalized'].mean()

# Calculate mean of original UV_2_260_mAU for both groups (as requested by plan
    feedback)
raw_mean_with_ph = data_with_ph['UV_2_260_mAU'].mean()
raw_mean_without_ph = data_without_ph['UV_2_260_mAU'].mean()

# Perform t-test for selection bias
t_stat, p_value = stats.ttest_ind(data_with_ph['UV_2_260_mAU_normalized'],
    data_without_ph['UV_2_260_mAU_normalized'])

# Plot 1: Boxplot of normalized UV_2_260_mAU split by pH_pH availability
plt.figure(figsize=(10, 6))
plt.boxplot([data_with_ph['UV_2_260_mAU_normalized'], data_without_ph['
    UV_2_260_mAU_normalized']], labels=['pH_Present', 'pH_Missing'])
plt.title('Boxplot of Normalized UV_2_260_mAU by pH_pH Availability')
plt.ylabel('Normalized Signal')
plt.xlabel('pH_pH Availability')
plt.tight_layout()
plt.savefig('/workspace/boxplot_normalized_uv.png')
plt.close()

# Plot 2: Scatter plot of Fraction_number vs UV_2_260_mAU colored by pH_pH
    availability
plt.figure(figsize=(10, 6))
scatter = plt.scatter(data['Fraction_number'], data['UV_2_260_mAU_normalized'],
    c=data['pH_pH'].notna().astype(int), cmap='viridis', alpha
    =0.6)

```

```

plt.colorbar(scatter, label='pH_pH_Availability_(1=Present, 0=Missing)')
plt.title('Scatter Plot: Fraction_number vs Normalized UV_2_260_mAU')
plt.xlabel('Fraction_number')
plt.ylabel('Normalized UV_2_260_mAU')
plt.tight_layout()
plt.savefig('/workspace/scatter_fraction_uv.png')
plt.close()

# Create markdown summary
summary = f"""
## Signal Quality and Missingness Analysis Summary

- **Percentage of missing #pH_pH#:** {pH_missing_pct:.2f}%
- **t-test p-value (Selection Bias):** {p_value:.2e}
- **Mean Normalized Signal (pH Present):** {mean_with_ph:.4f}
- **Mean Normalized Signal (pH Missing):** {mean_without_ph:.4f}
- **Mean Raw UV_2_260_mAU (pH Present):** {raw_mean_with_ph:.4f}
- **Mean Raw UV_2_260_mAU (pH Missing):** {raw_mean_without_ph:.4f}
- **Calculated SNR:** {snr:.4f}
"""

# Save summary to a markdown file
with open('/workspace/analysis_summary.md', 'w') as f:
    f.write(summary)

print("Analysis complete. Files saved to /workspace/")
###

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns
df = df[['run_no', 'UV_2_260_mAU', 'Conc_B_%']]

# Calculate global mean UV value across all runs
global_mean_uv = df['UV_2_260_mAU'].mean()

# Group by run and calculate mean UV for each run
run_means = df.groupby('run_no')['UV_2_260_mAU'].mean()

# Calculate distance as the absolute difference from the global mean
# This avoids the shape mismatch error by comparing scalars
distances = np.abs(run_means - global_mean_uv)

# Create dataframe for plotting
distances_df = pd.DataFrame({'run_no': run_means.index, 'distance': distances})

# Plot absolute difference of run means from global mean UV
plt.figure(figsize=(10, 6))
sns.boxplot(data=distances_df, x='run_no', y='distance')
plt.title('Absolute Difference of Run Means from Global Mean UV')
plt.xlabel('Run Number')

```



```

plt.ylabel('Distance from Global Mean')
plt.tight_layout()
plt.savefig('/workspace/euclidean_distance_plot.png')
plt.close()

# Calculate correlation with lags (-5 to +5) using vectorized shift
max_lag = 5
lags = np.arange(-max_lag, max_lag + 1)

mean_uv_series = df.groupby('run_no')['UV_2_260_mAU'].transform('mean')
correlations = []
for lag in lags:
    if lag == 0:
        corr_val = 1.0
    else:
        shifted_mean = mean_uv_series.shift(lag)
        corr_val = df['UV_2_260_mAU'].corr(shifted_mean)
    correlations.append({'lag': lag, 'correlation': corr_val})

correlations_df = pd.DataFrame(correlations)

# Find peak correlation
peak_corr = correlations_df.loc[correlations_df['correlation'].idxmax()]

# Generate heatmap
heatmap_data = correlations_df.set_index('lag')['correlation'].values.reshape(1,
-1)

plt.figure(figsize=(10, 8))
sns.heatmap(heatmap_data, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Heatmap: UV Signal vs Mean UV Trace (Lags -5 to +5)')
plt.xlabel('Lag')
plt.ylabel('Correlation Coefficient')
plt.tight_layout()
plt.savefig('/workspace/correlation_heatmap.png')
plt.close()

# Plot Euclidean distance grouped by run_no
plt.figure(figsize=(10, 6))
sns.boxplot(data=distances_df, x='run_no', y='distance')
plt.title('Euclidean Distance of Individual Runs from Mean Trace')
plt.xlabel('Run Number')
plt.ylabel('Distance')
plt.tight_layout()
plt.savefig('/workspace/euclidean_distance_plot.png')
plt.close()

```

Listing 3: Code_3